2026-8-17 Linux线程--互斥

- 1. 从抢票程序看线程互斥
  - 1.1 异常现象：同一张票可能被卖出多次
  - 1.2 线程共享资源与数据竞争
- 2. 数据不一致的根源
  - 2.1 临界资源、临界区、互斥与原子性
  - 2.2 条件判断与票数修改都不是完整的原子操作
    - 2.2.1 多个线程会同时通过余票判断
    - 2.2.2 usleep 放大了切换窗口
- 3. 用互斥锁保护临界区
  - 3.1 互斥保护需要满足的条件
  - 3.2 加锁位置与加锁粒度
  - 3.3 临界区中可以发生线程切换
  - 3.4 加锁失败的线程在做什么
- 4. 互斥量 mutex 的接口
  - 4.1 初始化互斥量
  - 4.2 加锁与解锁
  - 4.3 销毁互斥量
  - 4.4 加锁版抢票程序
- 5. 互斥锁的底层原理
  - 5.1 用原子交换竞争锁状态
  - 5.2 锁的本质是独占一份资源
  - 5.3 为什么不在用户态直接关闭中断
- 6. 用 RAII 封装互斥量
  - 6.1 封装 Mutex 类
  - 6.2 用 LockGuard 自动解锁
  - 6.3 RAII 版抢票代码
- 7. 线程互斥的核心结论

1. 从抢票程序看线程互斥

多线程可以并发执行，并不意味着多个线程可以任意地并发修改同一份数据。下面用多线程售票程序模拟多个执行流同时抢票：

```
#include <iostream>
#include <unistd.h>
#include <pthread.h>
```

```c
int tickets = 100; // 所有线程共享这份票数

void* route(void* args)
{
    char* id = (char*)args;

    while (1) {
        if (tickets > 0) {
            usleep(1000); // 模拟抢票业务耗时，主动给调度器切换机会
            printf("%s get tickets: %d\n", id, tickets);
            tickets--;
        } else {
            break;
        }
    }
    return nullptr;
}

int main()
{
    pthread_t t1, t2, t3, t4;
    pthread_create(&t1, nullptr, route, (void*)"thread 1");
    pthread_create(&t2, nullptr, route, (void*)"thread 2");
    pthread_create(&t3, nullptr, route, (void*)"thread 3");
    pthread_create(&t4, nullptr, route, (void*)"thread 4");

    pthread_join(t1, nullptr);
    pthread_join(t2, nullptr);
    pthread_join(t3, nullptr);
    pthread_join(t4, nullptr);
    return 0;
}
```

1.1 异常现象：同一张票可能被卖出多次

按照直觉，票数减到 `0` 后所有线程都应该停止。但程序运行后会出现负数，说明一张“最后余票”曾被多个线程同时认定为可以出售：

```
thread 1 get tickets: 8
thread 2 get tickets: 7
thread 3 get tickets: 6
thread 4 get tickets: 5
thread 1 get tickets: 4
thread 2 get tickets: 3
thread 3 get tickets: 2
thread 4 get tickets: 1
thread 1 get tickets: 0
thread 2 get tickets: -1
thread 3 get tickets: -2
```

问题的入口不是 `printf` 的输出顺序，而是 `tickets` 是线程共享的全局变量。同一进程内的线程共享地址空间，所以一个线程修改 `tickets` 后，其他线程也能看到；如果多个线程在没有任何协调的情况下读写它，就会形成数据竞争。

1.2 线程共享资源与数据竞争

多个线程同时对显示器写入会导致输出交错，但这只是表面现象。更关键的是，售票逻辑中的“检查余票”和“减少票数”被拆成了多个 CPU 步骤。线程可能在任意两个步骤之间被切换，于是多个线程会基于同一份旧数据分别作出判断和修改。

因此要解决的不是“让屏幕打印不交错”，而是保证共享票数的**判断与修改过程**不会与其他线程交叉执行。

2. 数据不一致的根源

2.1 临界资源、临界区、互斥与原子性

讨论加锁之前，先明确四个概念：

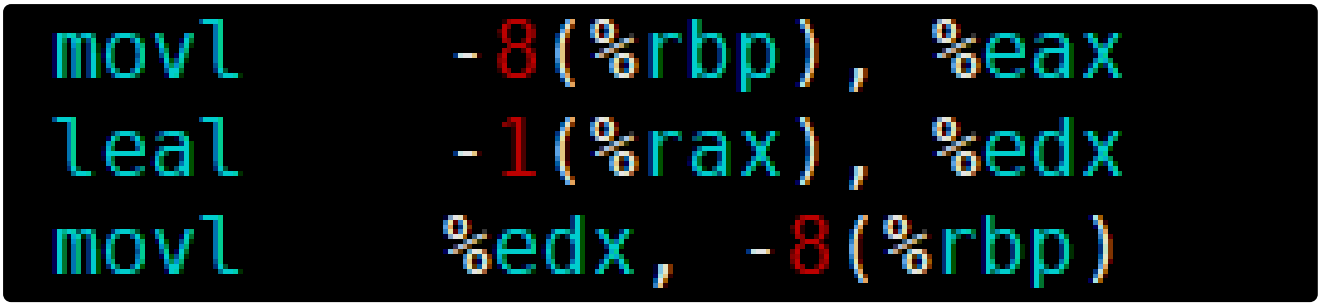
- **共享资源**：可以被多个线程共同访问的数据或对象，例如这里的全局变量 `tickets`。
- **临界资源**：需要被保护的共享资源。并发访问它时，如果缺少协调会产生数据不一致。
- **临界区**：每个线程中访问临界资源的那段代码。这里既包括 `tickets--`，也包括对 `tickets` 的判断。
- **互斥**：任意时刻至多允许一个执行流进入临界区。线程进入临界区前取得访问资格，离开临界区后再释放。
- **原子性**：一个操作不会被调度机制从中间打断，对外只呈现“尚未完成”或“已经完成”两种状态。

“共享资源”和“临界资源”不是按变量类型区分的。一个普通变量是否属于临界资源，取决于它是否被多个线程并发访问，以及并发访问是否会破坏程序语义。

2.2 条件判断与票数修改都不是完整的原子操作

`tickets--` 虽然只写了一行 C/C++ 代码，但编译后并不一定由一条 CPU 指令完成。它可以抽象为三个关键步骤：

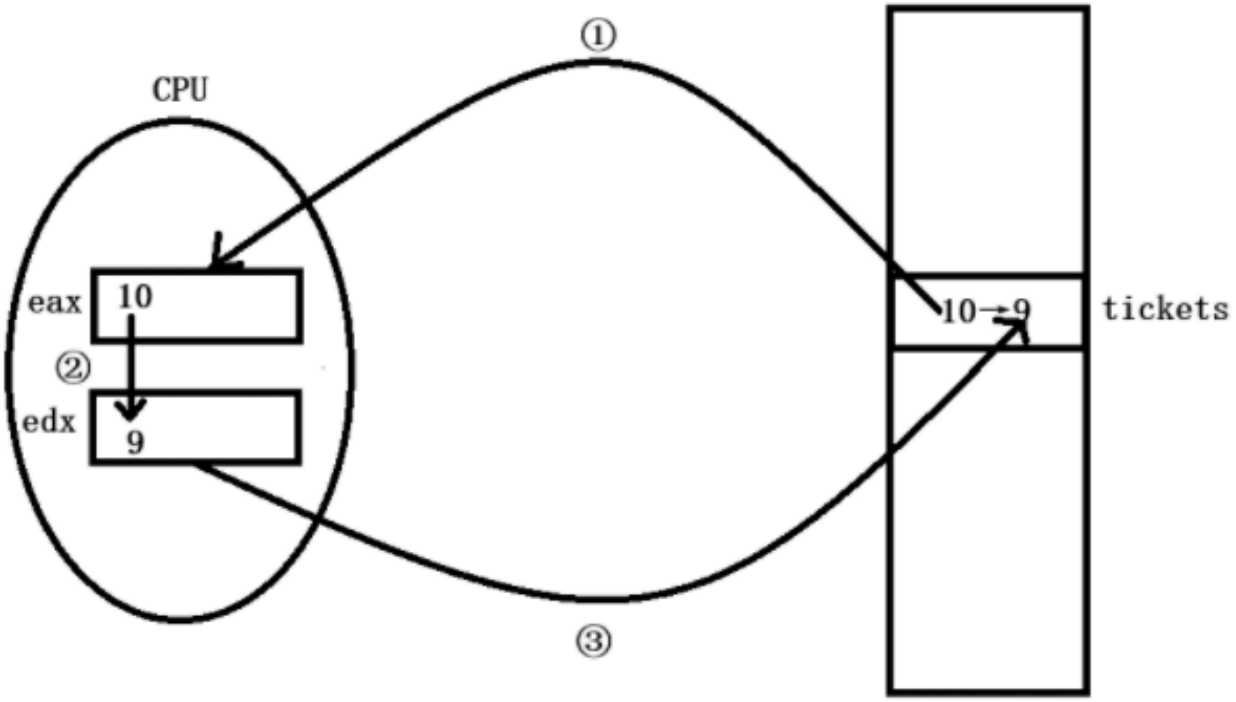
1. `load`：把共享变量 `tickets` 从内存加载到寄存器。
2. `update`：在寄存器中执行减一。
3. `store`：把新值写回 `tickets` 所在的内存地址。



如果把上述过程理解为一次“读取 - 修改 - 写回”，就很容易看出竞争窗口：

```
线程 A：读取 tickets = 100
线程 B：读取 tickets = 100
线程 A：计算 99，并写回 tickets = 99
线程 B：计算 99，并写回 tickets = 99
```

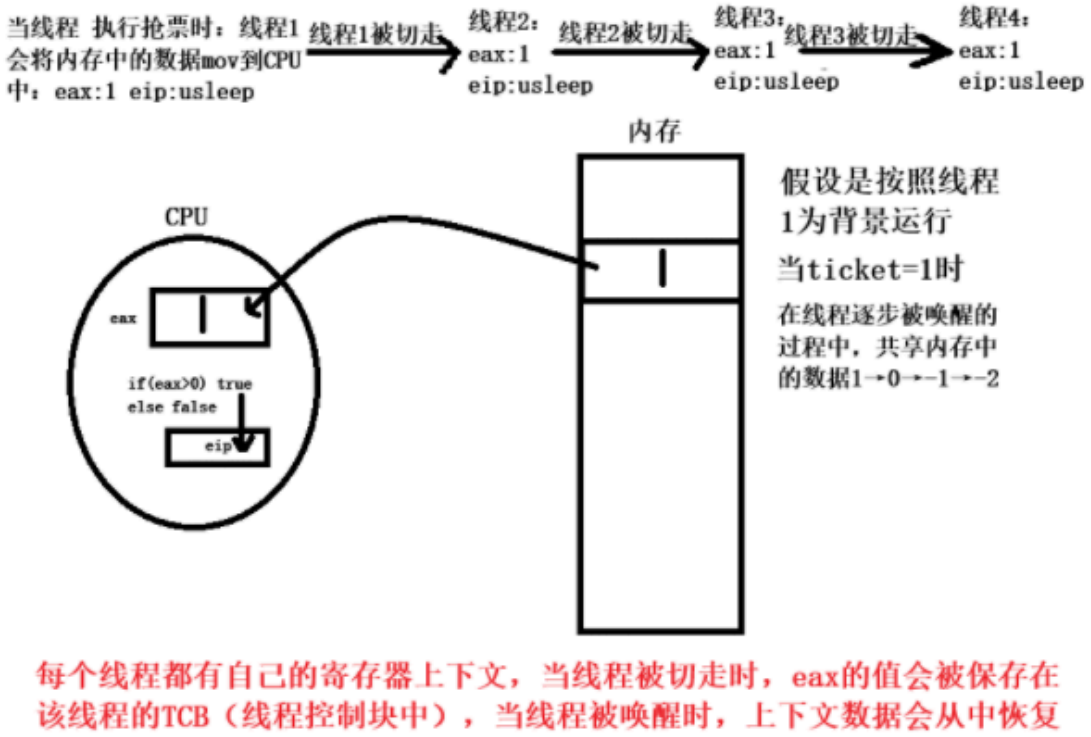
线程 A 的减一被线程 B 用旧值覆盖，这就是一次“丢失更新”。图中的三条机器指令分别对应加载、减一和写回：



2.2.1 多个线程会同时通过余票判断

更直接导致负数的是 `if (tickets > 0)` 本身也需要保护。假设内存中只剩 `tickets = 1`，线程 1 读取到 `1` 后通过判断，进入 `if` 语句并执行 `usleep`；在它阻塞期间，线程 2、3、4 也可能依次读取到同一个 `1` 并通过判断。

```c
void* route(void* args)
{
    //类型转化用以接收函数
    char* id =(char*) args;
    while(1){
        if(tickets>0)
        {
            usleep(1000);//模拟具体抢票花的时间
            printf("%s get tickets: %d\n",id,tickets);
            tickets--;
        }
        else
        {
            break;
        }
    }
    return nullptr;
}
```



此时四个线程都已经取得了“最后一张票仍然存在”的判断结果，各自保存着自己的寄存器上下文。它们被唤醒后依次恢复执行：第一个线程把 `1` 减成 `0`，后面的线程仍会按照先前通过的判断继续售卖，于是可能出现 `-1`、`-2`。

这个例子说明了一个容易被忽略的点：**判断结果和基于该结果执行的操作必须处于同一个临界区中**。只保护 `tickets--` 而把 `if (tickets > 0)` 留在锁外，仍然可能出现多个线程同时通过判断。

2.2.2 `usleep` 放大了切换窗口

`usleep(1000)` 不只是“等待一毫秒”，它会让当前线程进入可睡眠状态，调度器可以切换到其他线程。线程被挂起时，当前的程序计数器和寄存器现场会被保存；恢复运行时再从保存的上下文中继续。

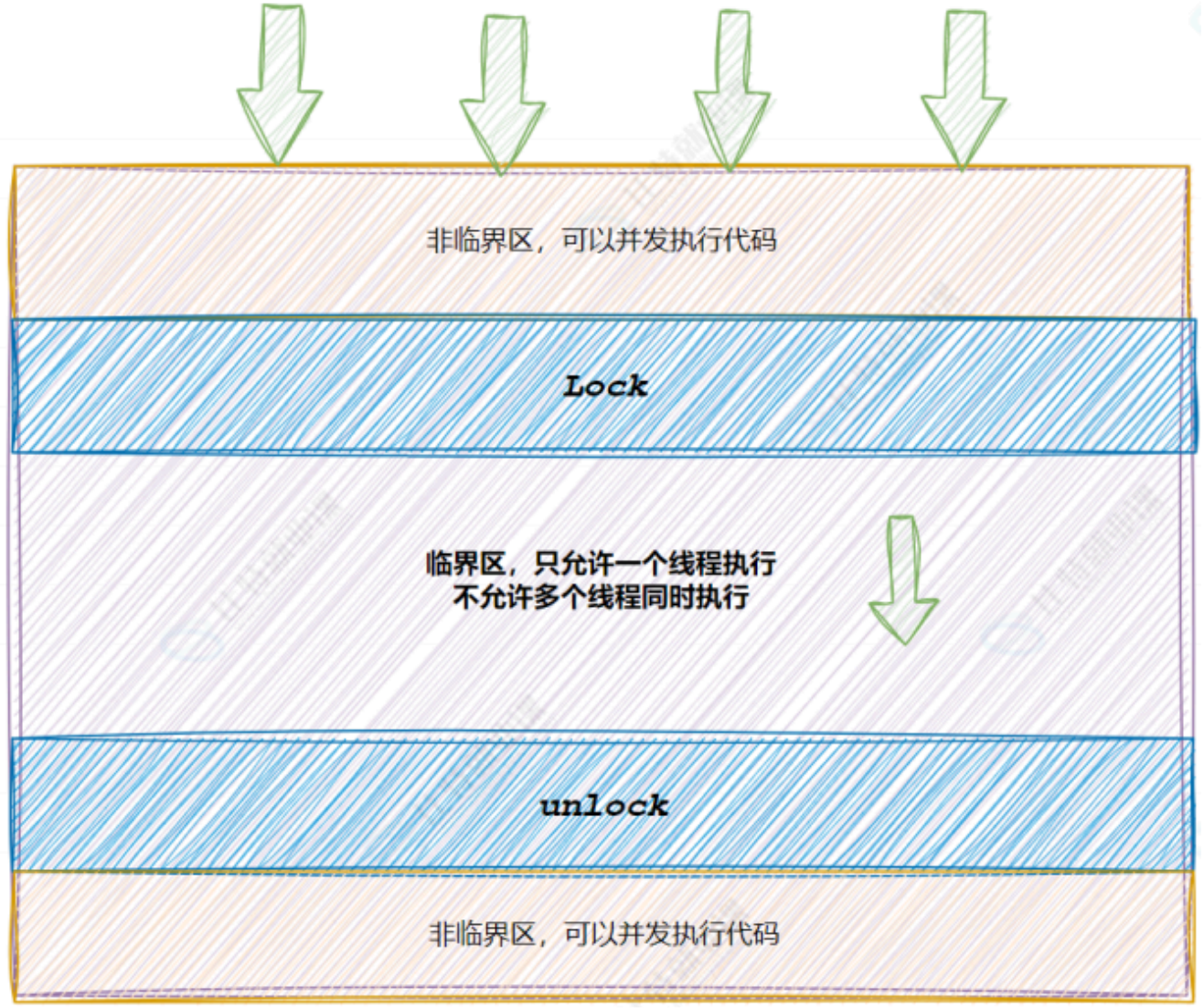
所以 `usleep` 只是人为放大了竞争窗口。真正的问题在于，即使没有 `usleep`，线程也可能在时间片耗尽时被切换，不能依赖“某几条语句执行得足够快”来保证安全。

3. 用互斥锁保护临界区

3.1 互斥保护需要满足的条件

要让多个线程安全地共享票数，锁必须满足以下三点：

1. 一个线程进入临界区后，其他线程不能同时进入同一临界区。
2. 临界区空闲，并且有多个线程同时申请进入时，只能让其中一个线程进入。
3. 没有进入临界区的线程，不能阻止其他线程进入。



把这三条要求落实到代码中，就是在临界区前后成对执行 `lock` 和 `unlock`：

```
非临界区：可以并发执行
        ↓
```

```
      lock
        ↓
临界区：只允许一个线程执行
        ↓
     unlock
        ↓
非临界区：可以继续并发执行
```

保护临界资源并不是给变量本身加一个特殊属性，而是要求所有访问它的线程共同遵守规则：进入临界区前加锁，离开临界区时解锁。

3.2 加锁位置与加锁粒度

锁的位置决定了临界区的大小。锁放得太早，会把原本可以并发执行的代码串行化；锁放得太晚，又可能让共享数据的判断和修改分处锁内外。

合理的做法是：

- 在第一次读取共享状态之前加锁。
- 把“判断余票、打印、修改票数”等构成一次完整业务操作的部分放在同一个临界区中。
- 在离开临界区时解锁，并保证 `if` 的每个退出分支都能解锁。
- 所有访问同一份共享资源的线程必须使用同一个互斥量。

锁的粒度应尽量细，但前提是临界区完整覆盖了需要原子执行的共享访问。只追求“锁得少”而破坏原子性，仍然会产生数据竞争。

3.3 临界区中可以发生线程切换

加锁并不要求在临界区内关闭调度，也不要求线程在这段时间内一直占用 CPU。线程持有锁时仍然可能因为时间片耗尽而被切换走。

只要它没有执行 `unlock`，这把锁就一直由它持有；其他线程虽然可以在处理器上运行，却会在 `pthread_mutex_lock` 处被阻塞，无法进入同一临界区。因此，线程切换本身不会破坏互斥，真正需要保证的是：**锁的取得与释放不能被并发操作打断**。

3.4 加锁失败的线程在做什么

当线程申请一把已经被占用的普通互斥锁时，它会在锁上阻塞等待，不会继续执行临界区。持锁线程解锁并唤醒等待者后，等待线程重新尝试获取锁；如果此时又有其他线程竞争，仍然只

能有一个线程成功。

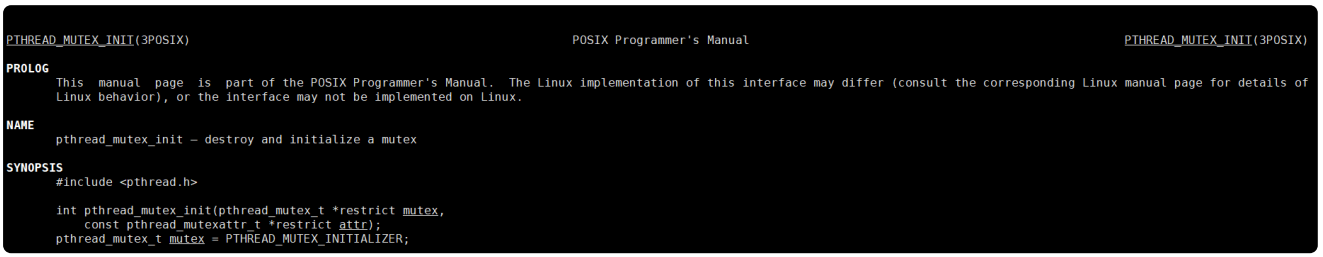
```
申请加锁失败
      ↓
阻塞等待
      ↓
持锁线程 unlock
      ↓
被唤醒后重新尝试竞争
      ↓
成功者进入临界区
```

这也说明锁是一种所有相关线程共同遵守的协议。只要有一个线程绕过加锁直接修改共享数据，协议就被破坏，其他线程遵守规则也不能保证最终结果正确。

4. 互斥量 mutex 的接口

Linux 把这种用于互斥的锁称为互斥量（mutex），类型是 `pthread_mutex_t`。它需要被所有相关线程看见，通常放在全局区，或者由多个线程共享同一个对象。

4.1 初始化互斥量



```
PTHREAD_MUTEX_INIT(3POSIX)                    POSIX Programmer's Manual                    PTHREAD_MUTEX_INIT(3POSIX)

PROLOG
       This  manual  page  is  part of the POSIX Programmer's Manual.  The Linux implementation of this interface may differ (consult the corresponding Linux manual page for details of
       Linux behavior), or the interface may not be implemented on Linux.

NAME
       pthread_mutex_init – destroy and initialize a mutex

SYNOPSIS
       #include <pthread.h>

       int pthread_mutex_init(pthread_mutex_t *restrict mutex,
           const pthread_mutexattr_t *restrict attr);
       pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
```

静态分配的互斥量可以在定义时直接初始化：

```c
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
```

这种方式适用于全局或静态存储期的互斥量，不需要再单独调用初始化函数。

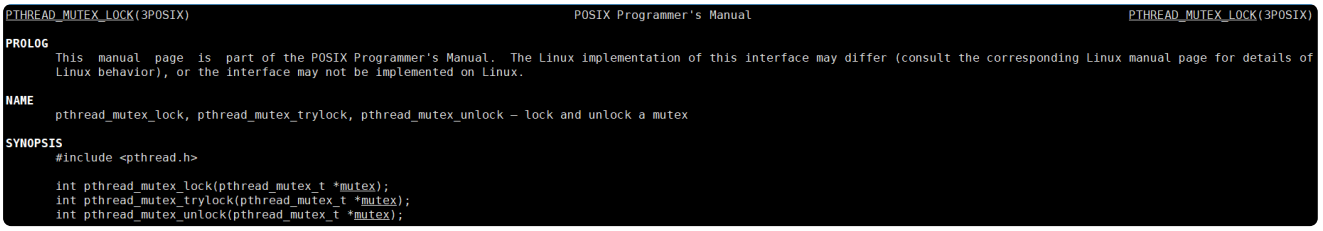
动态初始化使用 `pthread_mutex_init`：

```c
int pthread_mutex_init(
    pthread_mutex_t *restrict mutex,
    const pthread_mutexattr_t *restrict attr);
```

- `mutex`：要初始化的互斥量。
- `attr`：互斥量属性，传 `nullptr` 或 `NULL` 表示使用默认属性。
- 返回值：成功返回 `0`，失败返回错误码。

多个线程共享同一把锁，并不意味着每个线程都能拥有锁；同一把锁只有一个“已加锁/未加锁”状态，所有线程都在竞争这一份状态。

4.2 加锁与解锁



```
PTHREAD_MUTEX_LOCK(3POSIX)                POSIX Programmer's Manual                PTHREAD_MUTEX_LOCK(3POSIX)

PROLOG
       This  manual  page  is  part of the POSIX Programmer's Manual.  The Linux implementation of this interface may differ (consult the corresponding Linux manual page for details of
       Linux behavior), or the interface may not be implemented on Linux.

NAME
       pthread_mutex_lock, pthread_mutex_trylock, pthread_mutex_unlock — lock and unlock a mutex

SYNOPSIS
       #include <pthread.h>

       int pthread_mutex_lock(pthread_mutex_t *mutex);
       int pthread_mutex_trylock(pthread_mutex_t *mutex);
       int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

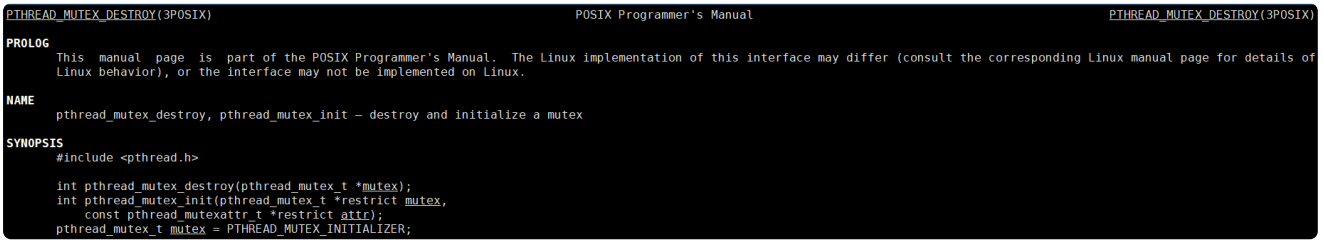
```c
int pthread_mutex_lock(pthread_mutex_t *mutex);
int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

`pthread_mutex_lock` 的行为可以概括为：

- 锁空闲时，当前线程取得锁，并从函数返回。
- 锁已被其他线程持有时，当前线程阻塞，直到锁被释放后再次参与竞争。
- 成功返回 `0`，失败返回错误码。

普通互斥量应在取得它的同一个线程中解锁。一个线程不能在未持有锁的情况下解锁另一个线程持有的普通互斥量。

4.3 销毁互斥量



```
PTHREAD_MUTEX_DESTROY(3POSIX)             POSIX Programmer's Manual             PTHREAD_MUTEX_DESTROY(3POSIX)

PROLOG
       This  manual  page  is  part of the POSIX Programmer's Manual.  The Linux implementation of this interface may differ (consult the corresponding Linux manual page for details of
       Linux behavior), or the interface may not be implemented on Linux.

NAME
       pthread_mutex_destroy, pthread_mutex_init — destroy and initialize a mutex

SYNOPSIS
       #include <pthread.h>

       int pthread_mutex_destroy(pthread_mutex_t *mutex);
       int pthread_mutex_init(pthread_mutex_t *restrict mutex,
           const pthread_mutexattr_t *restrict attr);
       pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
```

```c
int pthread_mutex_destroy(pthread_mutex_t *mutex);
```

销毁互斥量时要注意：

- 使用 `PTHREAD_MUTEX_INITIALIZER` 静态初始化的互斥量不需要手动销毁。
- 不要销毁仍在加锁状态的互斥量。
- 销毁前要确保之后不会再有线程访问或尝试加锁。
- 动态初始化的互斥量通常在所有工作线程都 `join` 完成后销毁。

4.4 加锁版抢票程序

下面的静态初始化版本把整个共享售票操作放进了临界区，并且保证两个分支都会解锁：

```
#include <iostream>
#include <unistd.h>
#include <pthread.h>

int tickets = 1000;
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

void* route(void* args)
{
    char* id = (char*)args;

    while (1) {
        pthread_mutex_lock(&mutex);
        if (tickets > 0) {
            usleep(1000);
            printf("%s get tickets: %d\n", id, tickets);
            tickets--;
            pthread_mutex_unlock(&mutex);
        } else {
            pthread_mutex_unlock(&mutex);
            break;
        }
    }
    return nullptr;
}

int main()
{
    pthread_t t1, t2, t3, t4;
    pthread_create(&t1, nullptr, route, (void*)"thread 1");
    pthread_create(&t2, nullptr, route, (void*)"thread 2");
    pthread_create(&t3, nullptr, route, (void*)"thread 3");
    pthread_create(&t4, nullptr, route, (void*)"thread 4");

    pthread_join(t1, nullptr);
```

```
    pthread_join(t2, nullptr);
    pthread_join(t3, nullptr);
    pthread_join(t4, nullptr);
    return 0;
}
```

这里有两个关键细节：

1. `pthread_mutex_lock` 必须放在循环内部。否则只有第一轮加锁，后续线程可能在没有锁保护的情况下继续执行。
2. `if` 的两个分支都必须解锁。进入 `else` 后也要先解锁再 `break`，否则一个线程退出时仍持有锁，其他线程可能永久等待。

如果采用动态初始化，主线程只需在创建工作线程之前初始化，并在所有线程结束后销毁：

```
pthread_mutex_t mutex;

int main()
{
    pthread_mutex_init(&mutex, nullptr);

    // 创建线程、pthread_join 等待所有线程结束......

    pthread_mutex_destroy(&mutex);
    return 0;
}
```

5. 互斥锁的底层原理

5.1 用原子交换竞争锁状态

可以把互斥量内部的关键状态简化成一个变量：

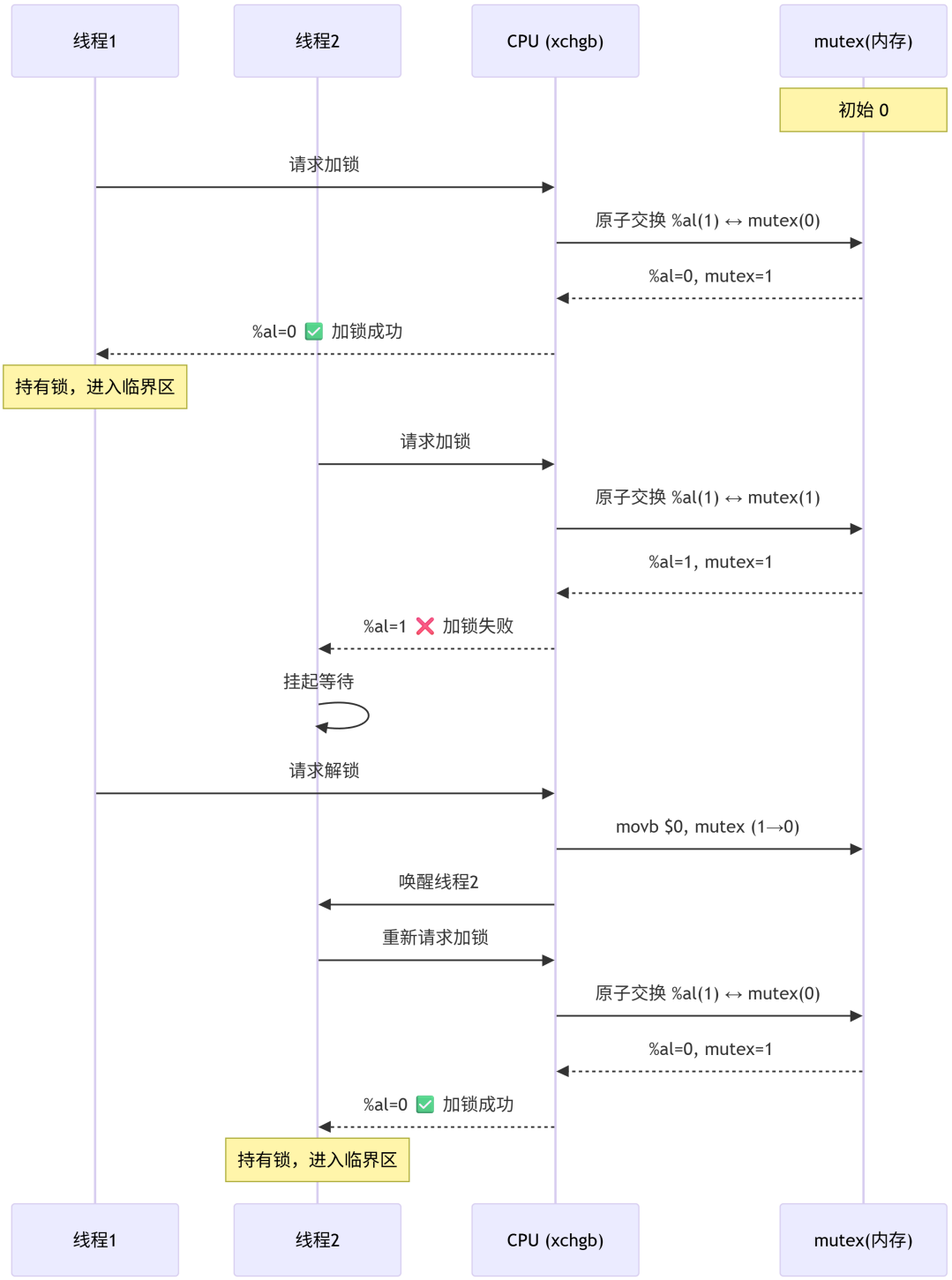
```
mutex = 0   表示锁空闲
mutex = 1   表示锁已被占用
```

很多处理器提供原子交换指令。以 `xchgb` 为例，它可以在一条硬件指令中交换寄存器和一个内存单元的值，中间不会再插入另一个线程的加锁操作。

加锁过程可以先这样理解：

```
lock:
    al = 1
    xchgb al, mutex
    如果交换前 mutex 为 0：
        al 得到 0，mutex 被写成 1，加锁成功
    如果交换前 mutex 为 1：
        al 得到 1，mutex 仍为 1，加锁失败
        挂起等待，之后重新尝试
```

`锁的原理.png` 展示了两个线程竞争同一把锁的完整时序：



线程 1 先把 `mutex` 从 `0` 交换成 `1`，所以它拿到旧值 `0` 并成功进入临界区。线程 2 随后发起交换，拿到旧值 `1`，说明锁已经被占用，只能挂起等待。线程 1 解锁后把 `mutex` 写回 `0`，线程 2 被唤醒并重新尝试加锁。

> 判断方向要以状态图为准：交换后 `al == 0` 表示加锁成功，`al == 1` 表示锁已被占用并需要等待。

原子交换解决的是“谁抢到了唯一一份锁状态”的问题；锁状态本身只需要一份，所以多个线程竞争时只能有一个线程成功取得它。

5.2 锁的本质是独占一份资源

从资源管理角度看，互斥锁可以理解成只有一份资源的信号量：

```
资源总量 = 1
取得资源 = 加锁
归还资源 = 解锁
```

申请人只有拿到这份唯一资源，才能进入临界区。这个类比有助于理解“互斥锁就是独占”，但两者在所有权、接口和使用规则上并不完全相同，不能把所有信号量都当成互斥锁使用。

互斥的保护对象始终是临界区。线程在临界区内访问共享变量，其他线程即使被调度到 CPU 上，也会因为拿不到同一把锁而停在临界区外。

5.3 为什么不在用户态直接关闭中断

关闭时钟中断确实可以暂时阻止当前处理器发生调度，但它属于内核级的特权操作。普通用户程序不能随意关闭中断，而且长时间关闭中断会影响整个系统，可能让其他任务都无法被及时调度。

互斥锁采用的是更细粒度的办法：不禁止所有线程运行，只让竞争同一临界区的线程在锁上等待。临界区外的代码仍然可以并发执行。

6. 用 RAII 封装互斥量

直接手写 `lock` 和 `unlock` 容易出现“中间某条路径提前返回而忘记解锁”的问题。C++ 可以用对象生命周期管理锁：构造时获取，析构时释放。

6.1 封装 Mutex 类

下面的封装把原始 `pthread_mutex_t` 隐藏起来，并禁止复制，避免两个对象错误地管理同一份锁状态：

```
#pragma once
```

```cpp
#include <pthread.h>

namespace LockModule {

class Mutex {
public:
    Mutex(const Mutex&) = delete;
    Mutex& operator=(const Mutex&) = delete;

    Mutex()
    {
        pthread_mutex_init(&mutex_, nullptr);
    }

    void Lock()
    {
        pthread_mutex_lock(&mutex_);
    }

    void Unlock()
    {
        pthread_mutex_unlock(&mutex_);
    }

    pthread_mutex_t* GetMutexOriginal()
    {
        return &mutex_;
    }

    ~Mutex()
    {
        pthread_mutex_destroy(&mutex_);
    }

private:
    pthread_mutex_t mutex_;
};

} // namespace LockModule
```

6.2 用 LockGuard 自动解锁

`LockGuard` 在构造时加锁，在析构时解锁。只要对象离开作用域，析构函数就会执行，因此即使代码通过 `break`、`return` 或异常离开当前作用域，也能释放锁：

```
class LockGuard {
public:
    explicit LockGuard(Mutex& mutex)
        : mutex_(mutex)
    {
        mutex_.Lock();
    }

    LockGuard(const LockGuard&) = delete;
    LockGuard& operator=(const LockGuard&) = delete;

    ~LockGuard()
    {
        mutex_.Unlock();
    }

private:
    Mutex& mutex_;
};
```

这种利用对象生命周期管理资源的做法称为 **RAII**（Resource Acquisition Is Initialization）。

6.3 RAII 版抢票代码

把 `LockGuard` 放进 `while` 循环体后，每一轮售票都会自动加锁，并在本轮作用域结束时自动解锁：

```
#include <iostream>
#include <unistd.h>
#include <pthread.h>
#include "Lock.hpp"

using LockModule::Mutex;
using LockModule::LockGuard;

int tickets = 1000;
Mutex mutex;

void* route(void* args)
{
    char* id = (char*)args;

    while (1) {
```

```
        LockGuard lockGuard(mutex);

        if (tickets > 0) {
            usleep(1000);
            printf("%s sells ticket: %d\n", id, tickets);
            tickets--;
        } else {
            break;
        }
    }
    return nullptr;
}

int main()
{
    pthread_t t1, t2, t3, t4;
    pthread_create(&t1, nullptr, route, (void*)"thread 1");
    pthread_create(&t2, nullptr, route, (void*)"thread 2");
    pthread_create(&t3, nullptr, route, (void*)"thread 3");
    pthread_create(&t4, nullptr, route, (void*)"thread 4");

    pthread_join(t1, nullptr);
    pthread_join(t2, nullptr);
    pthread_join(t3, nullptr);
    pthread_join(t4, nullptr);
    return 0;
}
```

`break` 离开 `while` 作用域时，`lockGuard` 会先析构并释放锁，所以不再需要像手写版本那样在每个分支分别调用 `pthread_mutex_unlock`。

7. 线程互斥的核心结论

- 多个线程共享地址空间，共享变量的读取、判断和修改都可能形成临界区。
- 一条 C/C++ 语句不一定是原子操作；`tickets--` 通常包含加载、修改和写回等多个步骤。
- 互斥锁通过保证同一临界区至多只有一个线程执行，消除判断与修改之间的竞争窗口。
- `lock` 和 `unlock` 必须成对出现，所有访问同一共享资源的线程必须遵守同一把锁的协议。
- 锁的底层关键是原子地竞争一份锁状态；线程在临界区内仍可被切换，但未解锁前其他线程不能进入同一临界区。
- 锁会降低并发度，因此应在保证临界区完整的前提下缩小加锁范围。

- C++ 中可用 `Mutex + LockGuard` 的 RAII 封装自动管理加锁与解锁，减少遗漏解锁的风险。